COMP-421 Database Systems

School of Computer Science, McGill University

Project 2: Creating and Working with Your Database

Hotel Booking Platform

Group 19
David Vo David Zhou David Namgung Jessie Maeda-Yang

Due: March 11, 2026

Contents

1 Relational Schema **3**

2 SQL Queries **5**
2.1 Query 1 5
2.2 Query 2 6
2.3 Query 3 9
2.4 Query 4 10
2.5 Query 5 12

3 SQL Modifications **15**
3.1 Modification 1 15
3.2 Modification 2 16

4 Views **18**
4.1 View 1: HotelSummary (Simple View) 18
4.2 View 2: ActiveReservationDetails (Complex View) 20

5 Check Constraints **22**
5.1 Check Constraint 1: Hotel Star Rating (`chk_star_rating`) 22
5.2 Check Constraint 2: Reservation Date Order (`chk_dates`) 23

6 Creativity **25**
6.1 Real-World Data Sets 25
6.2 Advanced SQL Feature: Window Functions 26

7 Group Work Organization **27**

1 Relational Schema

The relational schema for our Hotel Booking Platform is reproduced below from Project 1, incorporating all feedback received. The schema captures the full lifecycle of a hotel booking system including customers, hotels, room management, reservations, payments, reviews, staff, services, and amenities.

P1 Feedback Corrections

The following corrections were applied based on Project 1 grader feedback:

- **LoyaltyTier modelled as strong entity (not weak):** In the original P1 ER diagram, `LoyaltyTier` was incorrectly drawn as a weak entity dependent on `Customer`. Tiers such as Silver, Gold, and Platinum exist independently and are shared by many customers; they do not require a specific customer to exist. `LoyaltyTier` is therefore a **strong entity** with `tierName` as its own primary key, and `Customer.tierName` is a foreign key referencing `LoyaltyTier(tierName)`.
- **Room/RoomType/Hotel hotel-consistency:** The P1 ER diagram did not show an explicit constraint ensuring that a Room's hotel matches its RoomType's hotel. In the relational schema this is enforced structurally: `RoomType` has composite primary key `(hotelID, typeName)`, and `Room` carries a composite foreign key `(hotelID, typeName)` → `RoomType`. Because both the Room's own `hotelID` and the RoomType lookup key use the same `hotelID` column, the SQL schema guarantees that a room's hotel always matches its room type's hotel without requiring an additional constraint.
- **Service booking semantics:** `Service` represents optional add-on services offered by a hotel (e.g., spa, airport transfer, guided tour). A hotel exposes a catalogue of services via the `Service` table (`hotelID` FK). Guests may request services verbally at check-in; no separate booking junction table is modelled because service requests are handled operationally rather than transactionally in our schema scope.
- **Foreign key for tierName:** In the P1 written schema `Customer.tierName` was incorrectly described as referencing `Customer`. This has been corrected: `Customer.tierName` is a foreign key referencing `LoyaltyTier(tierName)`.

Schema Definition

LoyaltyTier(<u>tierName</u>, pointsThreshold, discountPercentage, specialBenefits)

Customer(<u>customerID</u>, fName, lName, email, phoneNumber, dateOfBirth, registrationDate, loyaltyPoints, tierName)
tierName foreign key referencing *LoyaltyTier*

Hotel(<u>hotelID</u>, hotelName, street, city, stateProvince, country, postalCode, starRating, phoneNumber, email, totalRooms, checkInTime, checkOutTime, propertyType)

RoomType(hotelID, typeName, description, maxOccupancy, bedConfiguration, sizeInSqMeters, basePricePerNight)
hotelID foreign key referencing *Hotel*

Room(roomID, roomNumber, floorNumber, roomStatus, hotelID, typeName)
hotelID foreign key referencing *Hotel*
(hotelID, typeName) foreign key referencing *RoomType*

Reservation(reservationID, checkInDate, checkOutDate, numberOfGuests, totalPrice, bookingDate, reservationStatus, specialRequests, cancellationPolicyType, customerID)
customerID foreign key referencing *Customer*

ReservationRoom(reservationID, roomID, pricePerNight)
reservationID foreign key referencing *Reservation*
roomID foreign key referencing *Room*

Payment(paymentID, paymentDate, amount, paymentMethod, transactionID, paymentStatus, reservationID)
reservationID foreign key referencing *Reservation*

Review(reviewID, rating, reviewText, reviewDate, cleanlinessRating, locationRating, serviceRating, valueRating, flaggedForModeration, customerID, hotelID)
customerID foreign key referencing *Customer*
hotelID foreign key referencing *Hotel*

Staff(staffID, fName, lName, email, phoneNumber, hireDate, salary, employmentStatus, hotelID)
hotelID foreign key referencing *Hotel*

Service(serviceID, serviceName, description, price, durationMinutes, availabilityStatus, hotelID)
hotelID foreign key referencing *Hotel*

Amenity(amenityID, amenityName, description)

HotelAmenity(hotelID, amenityID)
hotelID foreign key referencing *Hotel*
amenityID foreign key referencing *Amenity*

Pending Constraints

The following constraints are important to our domain but **cannot be expressed** in the DB2 CREATE TABLE syntax with CHECK constraints:

1. **No Double Booking:** A room cannot be reserved for overlapping date ranges. Formally: no two active reservations (status $\neq$ `'cancelled'`) may share the same `roomID` where the date ranges overlap (i.e., `checkInDate` of one reservation falls before `checkOutDate` of another and vice versa). This requires checking across multiple rows in `Reservation` joined with `ReservationRoom` and cannot be expressed in a single-tuple CHECK constraint.

2. **Loyalty Points Consistency:** A customer's `loyaltyPoints` must be at least as large as the `pointsThreshold` of their assigned `tierName`. For example, a `Gold` customer must have at least 5000 points. Enforcing this requires a multi-table comparison between `Customer.loyaltyPoints` and `LoyaltyTier.pointsThreshold`, which cannot be done with a simple CHECK constraint.
3. **Room Belongs to Reserved Hotel:** In `ReservationRoom`, the room's `hotelID` must match the hotel expected by the reservation. This cross-table constraint ensuring that rooms booked in a reservation all belong to the same hotel cannot be expressed with foreign keys alone.
4. **Service Availability for Reservation Hotel:** If a guest requests a service in connection with a reservation, that service must belong to the same hotel as the reservation. Since services are handled operationally (no `ReservationService` junction table), this cross-table rule cannot be expressed as a CHECK constraint.

2 SQL Queries

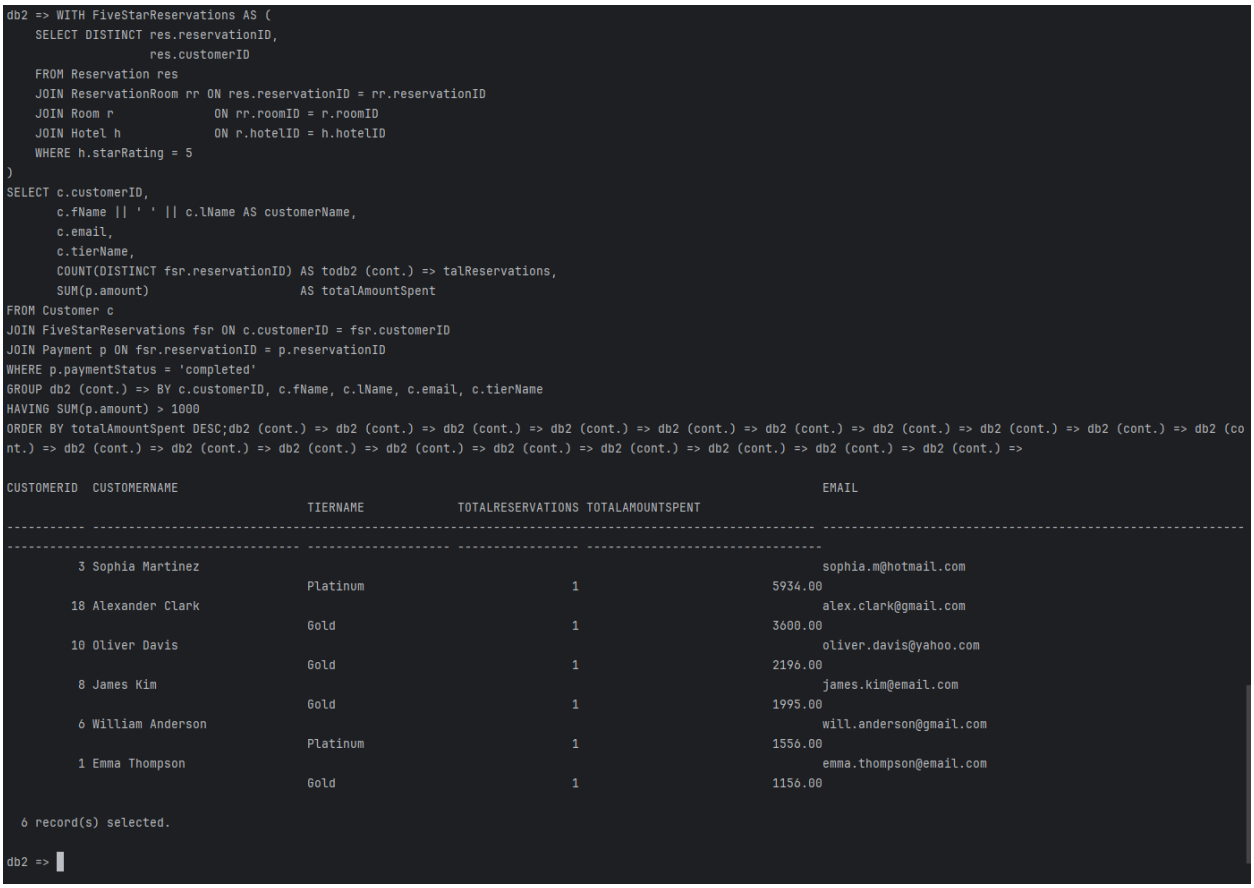
2.1 Query 1

Description: Find all customers who have made at least one reservation at a 5-star hotel, along with the total amount they have spent on completed payments and their current loyalty tier. This is useful for the hotel's VIP marketing team to identify high-value customers who prefer luxury properties.

```sql
WITH FiveStarReservations AS (
    SELECT DISTINCT res.reservationID,
                    res.customerID
    FROM Reservation res
    JOIN ReservationRoom rr ON res.reservationID = rr.reservationID
    JOIN Room r             ON rr.roomID = r.roomID
    JOIN Hotel h            ON r.hotelID = h.hotelID
    WHERE h.starRating = 5
)
SELECT c.customerID,
       c.fName || ' ' || c.lName AS customerName,
       c.email,
       c.tierName,
       COUNT(DISTINCT fsr.reservationID) AS totalReservations,
       SUM(p.amount)                     AS totalAmountSpent
FROM Customer c
JOIN FiveStarReservations fsr ON c.customerID = fsr.customerID
JOIN Payment p ON fsr.reservationID = p.reservationID
WHERE p.paymentStatus = 'completed'
GROUP BY c.customerID, c.fName, c.lName, c.email, c.tierName
HAVING SUM(p.amount) > 1000
```

```
ORDER BY totalAmountSpent DESC;
```

Listing 1: Query 1: High-value customers at 5-star hotels



```
db2 => WITH FiveStarReservations AS (
    SELECT DISTINCT res.reservationID,
                    res.customerID
    FROM Reservation res
    JOIN ReservationRoom rr ON res.reservationID = rr.reservationID
    JOIN Room r             ON rr.roomID = r.roomID
    JOIN Hotel h            ON r.hotelID = h.hotelID
    WHERE h.starRating = 5
)
SELECT c.customerID,
       c.fName || ' ' || c.lName AS customerName,
       c.email,
       c.tierName,
       COUNT(DISTINCT fsr.reservationID) AS todb2 (cont.) => talReservations,
       SUM(p.amount)                     AS totalAmountSpent
FROM Customer c
JOIN FiveStarReservations fsr ON c.customerID = fsr.customerID
JOIN Payment p ON fsr.reservationID = p.reservationID
WHERE p.paymentStatus = 'completed'
GROUP db2 (cont.) => BY c.customerID, c.fName, c.lName, c.email, c.tierName
HAVING SUM(p.amount) > 1000
ORDER BY totalAmountSpent DESC;db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (co
nt.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) =>

CUSTOMERID  CUSTOMERNAME                                                
```

```sql
            AVG(CAST(rv.serviceRating AS DECIMAL(5,2)))      AS avgService,
            AVG(CAST(rv.valueRating AS DECIMAL(5,2)))        AS avgValue,
            COUNT(rv.reviewID)                               AS reviewCount
    FROM Review rv
    GROUP BY rv.hotelID
),
SpaHotels AS (
    SELECT ha.hotelID
    FROM HotelAmenity ha
    JOIN Amenity a ON ha.amenityID = a.amenityID
    WHERE a.amenityName = 'Spa'
)
SELECT h.hotelName,
       h.city,
       h.starRating,
       DECIMAL(hr.avgOverall, 5, 2)      AS avgOverallRating,
       DECIMAL(hr.avgCleanliness, 5, 2) AS avgCleanliness,
       DECIMAL(hr.avgService, 5, 2)      AS avgService,
       hr.reviewCount,
       CASE WHEN sh.hotelID IS NOT NULL THEN 'Yes' ELSE 'No' END AS hasSpa
FROM Hotel h
JOIN HotelRatings hr ON h.hotelID = hr.hotelID
LEFT JOIN SpaHotels sh ON h.hotelID = sh.hotelID
ORDER BY hr.avgOverall DESC;
```

Listing 2: Query 2: Hotel review averages with amenity info

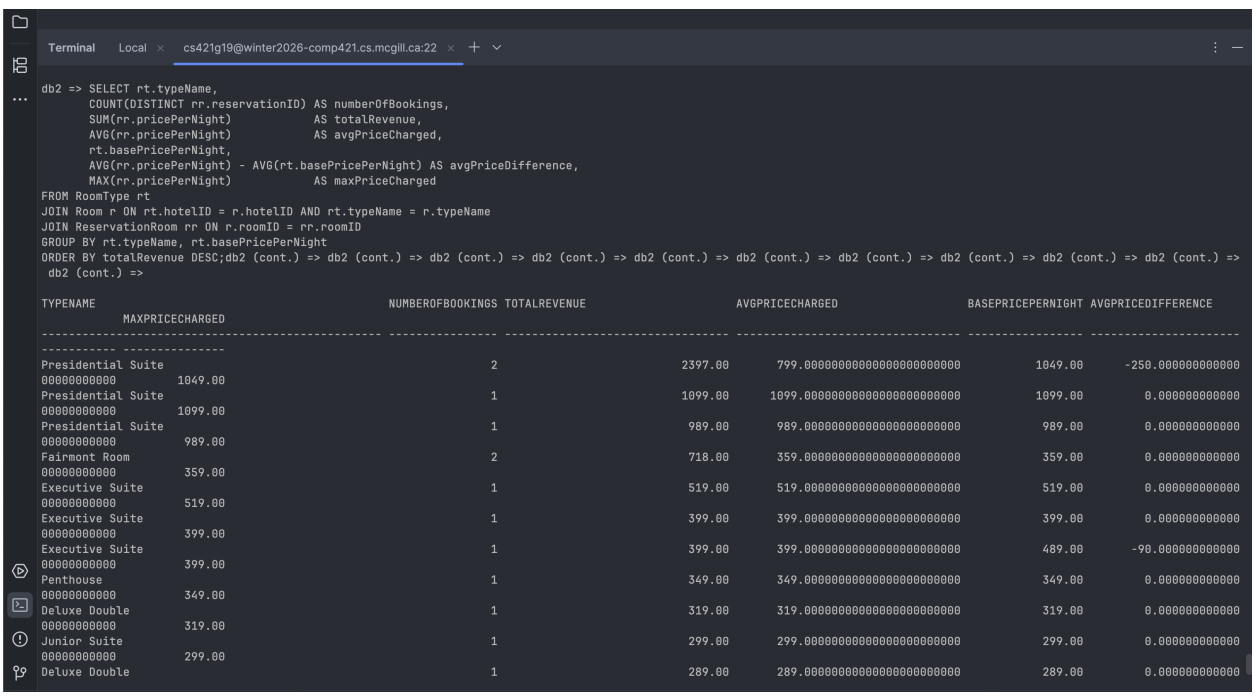
```
Terminal   Local ×   cs421g19@winter2026-comp421.cs.mcgill.ca:22 ×

db2 => WITH HotelRatings AS (
    SELECT rv.hotelID,
           AVG(CAST(rv.rating AS DECIMAL(5,2)))            AS avgOverall,
           AVG(CAST(rv.cleanlinessRating AS DECIMAL(5,2))) AS avgCleanliness,
           AVG(CAST(rv.locationRating AS DECIMAL(5,2)))    AS avgLocation,
           AVG(CAST(rv.serviceRating AS DECIMAL(5,2)))     AS avgService,
           AVG(CAST(rv.valueRating AS DECIMAL(5,2)))       AS avgValue,
           COUNT(rv.reviewID)                              AS reviewCount
    FROM Review rv
    GROUP BY rv.hotelID
),
SpaHotels AS (
    SELECT ha.hotelID
    FROM HotelAmenity ha
    JOIN Amenity a ON ha.amenityID = a.amenityID
    WHERE a.amenityName = 'Spa'
)
SELECT h.hotelName,
       h.city,
       h.stadb2 (cont.) => rRating,
       DECIMAL(hr.avgOverall, 5, 2)     AS avgOverallRating,
       DECIMAL(hr.avgCleanliness, 5, 2) AS avgCleanliness,
       DECIMAL(hr.avgService, 5, 2)     AS avgService,
       hr.reviewCount,
       CASE WHEN sh.hotelID IS NOT NULL THEN db2 (cont.) => 'Yes' ELSE 'No' END AS hasSpa
FROM Hotel h
JOIN HotelRatings hr ON h.hotelID = hr.hotelID
LEFT JOIN SpaHotels sh ON h.hotelID = sh.hotelID
ORDER BY hr.avgOverall DESC;db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) =
> db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont
.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) =>

HOTELNAME                                                                                                                CITY                                                                                                                
```

2.3 Query 3

Description: Find the most popular room types across all hotels, measured by number of bookings, total revenue generated, and average price per night charged. Use this information to identify which room categories drive the most revenue for the hotel chain.

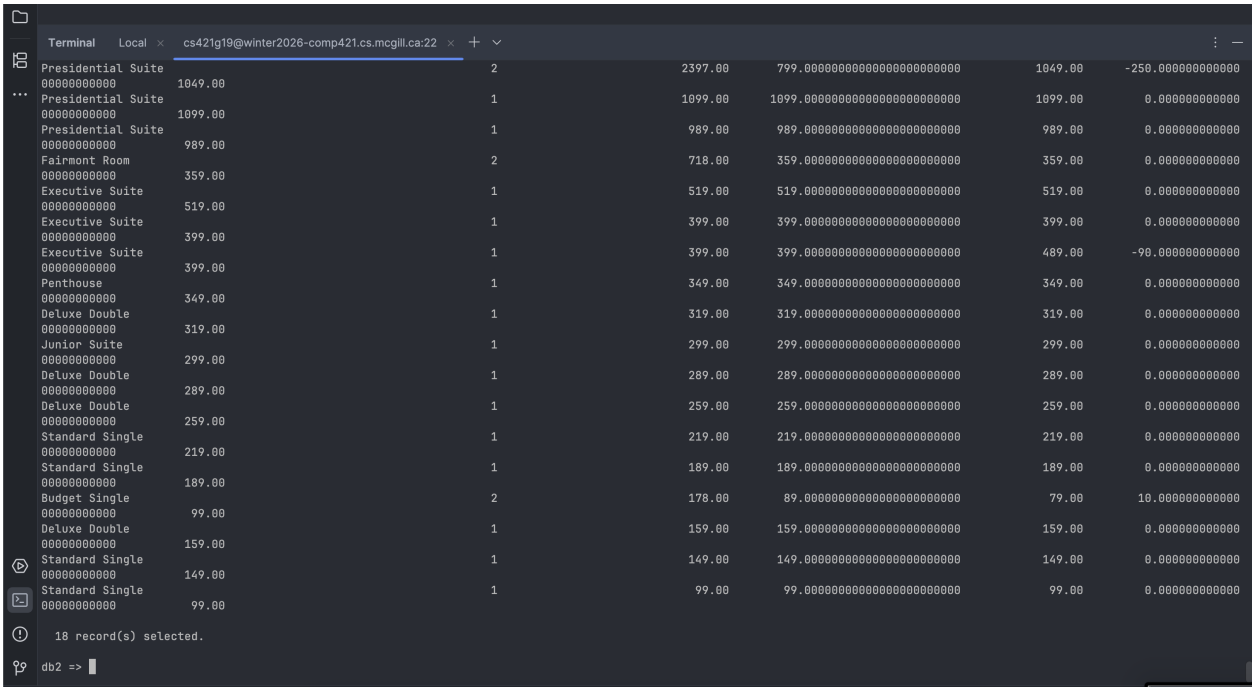
```sql
SELECT rt.typeName,
       COUNT(DISTINCT rr.reservationID) AS numberOfBookings,
       SUM(rr.pricePerNight)            AS totalRevenue,
       AVG(rr.pricePerNight)            AS avgPriceCharged,
       rt.basePricePerNight,
       AVG(rr.pricePerNight) - AVG(rt.basePricePerNight) AS
          avgPriceDifference,
       MAX(rr.pricePerNight)            AS maxPriceCharged
FROM RoomType rt
JOIN Room r ON rt.hotelID = r.hotelID AND rt.typeName = r.typeName
JOIN ReservationRoom rr ON r.roomID = rr.roomID
GROUP BY rt.typeName, rt.basePricePerNight
ORDER BY totalRevenue DESC;
```

Listing 3: Query 3: Revenue analysis by room type



```
Terminal   Local   cs421g19@winter2026-comp421.cs.mcgill.ca:22

db2 => SELECT rt.typeName,
       COUNT(DISTINCT rr.reservationID) AS numberOfBookings,
       SUM(rr.pricePerNight)            AS totalRevenue,
       AVG(rr.pricePerNight)            AS avgPriceCharged,
       rt.basePricePerNight,
       AVG(rr.pricePerNight) - AVG(rt.basePricePerNight) AS avgPriceDifference,
       MAX(rr.pricePerNight)            AS maxPriceCharged
FROM RoomType rt
JOIN Room r ON rt.hotelID = r.hotelID AND rt.typeName = r.typeName
JOIN ReservationRoom rr ON r.roomID = rr.roomID
GROUP BY rt.typeName, rt.basePricePerNight
ORDER BY totalRevenue DESC;db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) =>
 db2 (cont.) =>

TYPENAME                                           NUMBEROFBOOKINGS TOTALREVENUE                    AVGPRICECHARGED                  BASEPRICEPERNIGHT AVGPRICEDIFFERENCE
           MAXPRICECHARGED
-------------------------------------------------- ---------------- -------------------------------- -------------------------------- ----------------- ----------------------
---------- --------------
Presidential Suite                                                2                          2397.00      799.0000000000000000000000           1049.00      -250.000000000000
00000000000        1049.00
Presidential Suite                                                1                          1099.00     1099.0000000000000000000000           1099.00         0.000000000000
00000000000        1099.00
Presidential Suite                                                1                           989.00      989.0000000000000000000000            989.00         0.000000000000
00000000000         989.00
Fairmont Room                                                     2                           718.00      359.0000000000000000000000            359.00         0.000000000000
00000000000         359.00
Executive Suite                                                   1                           519.00      519.0000000000000000000000            519.00         0.000000000000
00000000000         519.00
Executive Suite                                                   1                           399.00      399.0000000000000000000000            399.00         0.000000000000
00000000000         399.00
Executive Suite                                                   1                           399.00      399.0000000000000000000000            489.00       -90.000000000000
00000000000         399.00
Penthouse                                                         1                           349.00      349.0000000000000000000000            349.00         0.000000000000
00000000000         349.00
Deluxe Double                                                     1                           319.00      319.0000000000000000000000            319.00         0.000000000000
00000000000         319.00
Junior Suite                                                      1                           299.00      299.0000000000000000000000            299.00         0.000000000000
00000000000         299.00
Deluxe Double                                                     1                           289.00      289.0000000000000000000000            289.00         0.000000000000
```

Figure 4: Query 3 output (part 1): Revenue analysis by room type



```
Terminal   Local   cs421g19@winter2026-comp421.cs.mcgill.ca:22
Presidential Suite                                  2          2397.00      799.00000000000000000000000000          1049.00     -250.000000000000
00000000000        1049.00
Presidential Suite                                  1          1099.00     1099.00000000000000000000000000          1099.00        0.000000000000
00000000000        1099.00
Presidential Suite                                  1           989.00      989.00000000000000000000000000           989.00        0.000000000000
00000000000         989.00
Fairmont Room                                       2           718.00      359.00000000000000000000000000           359.00        0.000000000000
00000000000         359.00
Executive Suite                                     1           519.00      519.00000000000000000000000000           519.00        0.000000000000
00000000000         519.00
Executive Suite                                     1           399.00      399.00000000000000000000000000           399.00        0.000000000000
00000000000         399.00
Executive Suite                                     1           399.00      399.00000000000000000000000000           489.00      -90.000000000000
00000000000         399.00
Penthouse                                           1           349.00      349.00000000000000000000000000           349.00        0.000000000000
00000000000         349.00
Deluxe Double                                       1           319.00      319.00000000000000000000000000           319.00        0.000000000000
00000000000         319.00
Junior Suite                                        1           299.00      299.00000000000000000000000000           299.00        0.000000000000
00000000000         299.00
Deluxe Double                                       1           289.00      289.00000000000000000000000000           289.00        0.000000000000
00000000000         289.00
Deluxe Double                                       1           259.00      259.00000000000000000000000000           259.00        0.000000000000
00000000000         259.00
Standard Single                                     1           219.00      219.00000000000000000000000000           219.00        0.000000000000
00000000000         219.00
Standard Single                                     1           189.00      189.00000000000000000000000000           189.00        0.000000000000
00000000000         189.00
Budget Single                                       2           178.00       89.00000000000000000000000000            79.00       10.000000000000
00000000000          99.00
Deluxe Double                                       1           159.00      159.00000000000000000000000000           159.00        0.000000000000
00000000000         159.00
Standard Single                                     1           149.00      149.00000000000000000000000000           149.00        0.000000000000
00000000000         149.00
Standard Single                                     1            99.00       99.00000000000000000000000000            99.00        0.000000000000
00000000000          99.00

  18 record(s) selected.

db2 =>
```

Figure 5: Query 3 output (part 2): Revenue analysis by room type (continued)

2.4 Query 4

Description: Identify customers whose loyalty tier does not match their total spending history – specifically customers in Bronze or Silver tiers who have spent more than the average spending of Platinum-tier customers. This helps the loyalty program team identify customers who should be upgraded.

```sql
WITH CustomerSpend AS (
    SELECT c.customerID,
           c.fName || ' ' || c.lName AS customerName,
           c.tierName,
           c.loyaltyPoints,
           COALESCE(SUM(p.amount), 0) AS totalSpent
    FROM Customer c
    LEFT JOIN Reservation res ON c.customerID = res.customerID
    LEFT JOIN Payment p ON res.reservationID = p.reservationID
                       AND p.paymentStatus = 'completed'
    GROUP BY c.customerID, c.fName, c.lName, c.tierName, c.
       loyaltyPoints
),
PlatinumAvg AS (
    SELECT AVG(totalSpent) AS platAvg
    FROM CustomerSpend
    WHERE tierName = 'Platinum'
)
SELECT cs.customerID,
```

```sql
        cs.customerName,
        cs.tierName,
        cs.loyaltyPoints,
        DECIMAL(cs.totalSpent, 12, 2)            AS totalSpent,
        DECIMAL(
            (SELECT platAvg FROM PlatinumAvg),
            12, 2
        )                                         AS
           platinumAvgSpend
FROM CustomerSpend cs
WHERE cs.tierName IN ('Bronze', 'Silver')
  AND cs.totalSpent > (SELECT platAvg FROM PlatinumAvg)
ORDER BY cs.totalSpent DESC;
```

Listing 4: Query 4: Loyalty tier mismatch detection

```
db2 => WITH CustomerSpend AS (
    SELECT c.customerID,
           c.fName || ' ' || c.lName AS customerName,
           c.tierName,
           c.loyaltyPoints,
           COALESCE(SUM(p.amount), 0) AS totalSpent
    FROM Customer c
    LEFT JOIN Reservation res ON c.customerID = res.customerID
    LEFT JOIN Payment p ON res.reservationID = p.reservationID
                       AND p.paymentStatus = 'completed'
    GROUP BY c.customerID, c.fName, c.lName, c.tierName, c.loyaltyPoints
),
PlatinumAvg AS (
    SELECT AVG(totalSpent) AS platAvg
    FROM CustomerSpend
    WHERE tierName = 'Platinum'
)
SELECT cs.customerID,
       cs.customerName,
       cs.tierName,
       cs.loyaltyPoints,
       DECIMAL(cs.totalSpent, 12, 2) AS db2 (cont.) => totalSpent,
       DECIMAL(pa.platAvg, 12, 2)    AS platinumAvgSpend
FROM CustomerSpend cs, PlatinumAvg pa
WHERE cs.tierName IN ('Bronze', 'Silver')
  AND cs.totalSpent > pa.platAvg
ORDER BY cs.totalSpent DESC;db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) =>
 db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.)
 => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) =>

CUSTOMERID  CUSTOMERNAME                                                                                                  TIERNAME            LOYALTYPOINT
S TOTALSPENT     PLATINUMAVGSPEND
----------- ---------------------------------------------------------------------------------------------------------- -------------------- ------------
```

Figure 6: Query 4 output (part 1): Loyalty tier mismatch detection

```
CUSTOMERID  CUSTOMERNAME                                                                                                  TIERNAME           LOYALTYPOINT
S TOTALSPENT     PLATINUMAVGSPEND
----------- ------------------------------------------------------------------------------------------------------------- ------------------- ------------
- -------------- ---------------
         16 Henry White                                                                                                   Silver                     2500
0        4945.00            0.00
         18 Alexander Clark                                                                                               Silver                      720
0        3600.00            0.00
         10 Oliver Davis                                                                                                  Silver                     1200
0        2196.00            0.00
          8 James Kim                                                                                                     Silver                      640
0        1995.00            0.00
          6 William Anderson                                                                                              Silver                     1850
0        1556.00            0.00
          5 Olivia Chen                                                                                                   Silver                      320
0        1445.00            0.00
          1 Emma Thompson                                                                                                 Silver                      875
0        1156.00            0.00
          4 Noah Patel                                                                                                    Bronze                       40
0         578.00            0.00
          9 Isabella Robinson                                                                                             Bronze                       10
0         447.00            0.00
          2 Liam Nguyen                                                                                                   Bronze                      120
0         298.00            0.00
          7 Ava Johnson                                                                                                   Bronze                       70
0         198.00            0.00

  11 record(s) selected.

db2 =>
```

Figure 7: Query 4 output (part 2): Loyalty tier mismatch detection (continued)

2.5 Query 5

Description: For each city where hotels are located, find the month with the highest booking revenue and compare it to the annual average monthly revenue. This seasonal analysis helps management determine peak periods for staffing and pricing adjustments.

```sql
WITH ReservationCity AS (
    -- Deduplicate: one row per (reservation, city) regardless of room count
    SELECT DISTINCT res.reservationID,
                    res.checkInDate,
                    h.city
    FROM Reservation res
    JOIN ReservationRoom rr ON res.reservationID = rr.reservationID
    JOIN Room r             ON rr.roomID = r.roomID
    JOIN Hotel h            ON r.hotelID = h.hotelID
),
MonthlyRevenue AS (
    SELECT rc.city,
           YEAR(rc.checkInDate)  AS bookingYear,
           MONTH(rc.checkInDate) AS bookingMonth,
           SUM(p.amount)         AS monthlyRevenue,
           COUNT(rc.reservationID) AS numBookings
    FROM ReservationCity rc
```

```sql
    JOIN Payment p ON rc.reservationID = p.reservationID
    WHERE p.paymentStatus = 'completed'
    GROUP BY rc.city, YEAR(rc.checkInDate), MONTH(rc.checkInDate)
),
CityAnnualAvg AS (
    SELECT city,
           bookingYear,
           AVG(monthlyRevenue) AS avgMonthlyRevenue
    FROM MonthlyRevenue
    GROUP BY city, bookingYear
),
RankedMonths AS (
    SELECT mr.city,
           mr.bookingYear,
           mr.bookingMonth,
           mr.monthlyRevenue,
           mr.numBookings,
           ca.avgMonthlyRevenue,
           RANK() OVER (PARTITION BY mr.city, mr.bookingYear
                        ORDER BY mr.monthlyRevenue DESC) AS revenueRank
    FROM MonthlyRevenue mr
    JOIN CityAnnualAvg ca ON mr.city = ca.city
                         AND mr.bookingYear = ca.bookingYear
)
SELECT city,
       bookingYear,
       bookingMonth AS peakMonth,
       DECIMAL(monthlyRevenue, 12, 2)    AS peakRevenue,
       DECIMAL(avgMonthlyRevenue, 12, 2) AS annualAvgMonthlyRevenue,
       DECIMAL(monthlyRevenue - avgMonthlyRevenue, 12, 2) AS aboveAverage,
       numBookings AS bookingsInPeakMonth
FROM RankedMonths
WHERE revenueRank = 1
ORDER BY city, bookingYear;
```

Listing 5: Query 5: Seasonal revenue analysis by city

```
db2 => WITH ReservationCity AS (
    -- Deduplicate: one row per (reservation, city) regardless of room count
    SELECT DISTINCT res.reservationID,
                    res.checkInDate,
                    h.city
    FROM Reservation res
    JOIN ReservationRoom rr ON res.reservationID = rr.reservationID
 db2 (cont.) =>    JOIN Room r              ON rr.roomID = r.roomID
    JOIN Hotel h             ON r.hotelID = h.hotelID
),
MonthlyRevenue AS (
    SELECT rc.city,
           YEAR(rc.checkInDate)  AS bookingYear,
           MONTH(rc.checkInDate) AS bookingMonth,
db2 (cont.) =>            SUM(p.amount)         AS monthlyRevenue,
           COUNT(rc.reservationID) AS numBookings
    FROM ReservationCity rc
    JOIN Payment p ON rc.reservationID = p.reservationID
    WHERE p.paymentStatus = 'completed'
    GROUP BY rc.city, YEAR(rc.checkInDate), MONTH(rc.checkInDate)
),
CityAnnualAvg AS (
    SELEdb2 (cont.) => CT city,
           bookingYear,
           AVG(monthlyRevenue) AS avgMonthlyRevenue
    FROM MonthlyRevenue
    GROUP BY city, bookingYear
),
RankedMonths AS (
    SELECT mr.city,
           mr.bookingYear,
           mr.bookingMonth,
           mr.monthlyRevenue,
           mr.numBookings,
   db2 (cont.) =>         ca.avgMonthlyRevenue,
           RANK() OVER (PARTITION BY mr.city, mr.bookingYear
                        ORDER BY mr.monthlyRevenuedb2 (cont.) =>  DESC) AS revenueRank
    FROM MonthlyRevenue mr
    JOIN CityAnnualAvg ca ON mr.city = ca.city
                          AND mr.bookingYedb2 (cont.) => ar = ca.bookingYear
)
SELECT city,
       bookingYear,
       bookingMonth AS peakMonth,
       DECIMAL(monthlyRevenue, 12, 2)   db2 (cont.) =>  AS peakRevenue,
       DECIMAL(avgMonthlyRevenue, 12, 2) AS annualAvgMonthlyRevenue,
       DECIMAL(monthlyRevenue - avgMonthlyRevenue, 12db2 (cont.) => , 2) AS aboveAverage,
       numBookings AS bookingsInPeakMonth
FROM RankedMonths
WHERE revenueRank = 1
ORDER BY city, bookingYear;db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.)
 => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2
(cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.)
=> db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) =>
```

Figure 8: Query 5 output (part 1): Seasonal revenue analysis by city

```
SELECT city,
       bookingYear,
       bookingMonth AS peakMonth,
       DECIMAL(monthlyRevenue, 12, 2)   db2 (cont.) =>  AS peakRevenue,
       DECIMAL(avgMonthlyRevenue, 12, 2) AS annualAvgMonthlyRevenue,
       DECIMAL(monthlyRevenue - avgMonthlyRevenue, 12db2 (cont.) => , 2) AS aboveAverage,
       numBookings AS bookingsInPeakMonth
FROM RankedMonths
WHERE revenueRank = 1
ORDER BY city, bookingYear;db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.)
 => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2
(cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.)
=> db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) =>

CITY                                               BOOKINGYEAR PEAKMONTH   PEAKREVENUE    ANNUALAVGMONTHLYREVENUE ABOVEAVERAGE   BOOKINGSINPEAKMONTH
-------------------------------------------------- ----------- ---------- -------------- ----------------------- -------------- -------------------
Montreal                                                  2026          2        7957.00                 5516.33        2440.67                   3
Quebec City                                               2026          3        7799.00                 7799.00           0.00                   4
Toronto                                                   2026          3        5198.00                 5198.00           0.00                   1

  3 record(s) selected.

db2 =>
```

Figure 9: Query 5 output (part 2): Seasonal revenue analysis by city (continued)

3 SQL Modifications

3.1 Modification 1

Description: Update the loyalty tier for all customers based on their total completed payment history. Customers who have spent at least $10,000 are upgraded to Platinum, those at least $5,000 to Gold, those at least $1,000 to Silver, and the rest remain Bronze. This batch update ensures that tier assignments reflect actual spending behaviour.

```sql
UPDATE Customer
SET tierName = CASE
        WHEN (SELECT COALESCE(SUM(p.amount), 0)
              FROM Reservation res
              JOIN Payment p ON res.reservationID = p.reservationID
              WHERE res.customerID = Customer.customerID
                AND p.paymentStatus = 'completed') >= 10000 THEN 'Platinum'
        WHEN (SELECT COALESCE(SUM(p.amount), 0)
              FROM Reservation res
              JOIN Payment p ON res.reservationID = p.reservationID
              WHERE res.customerID = Customer.customerID
                AND p.paymentStatus = 'completed') >= 5000 THEN 'Gold'
        WHEN (SELECT COALESCE(SUM(p.amount), 0)
              FROM Reservation res
              JOIN Payment p ON res.reservationID = p.reservationID
              WHERE res.customerID = Customer.customerID
                AND p.paymentStatus = 'completed') >= 1000 THEN 'Silver'
        ELSE 'Bronze'
    END
WHERE customerID IN (
    SELECT DISTINCT res.customerID
    FROM Reservation res
    JOIN Payment p ON res.reservationID = p.reservationID
    WHERE p.paymentStatus = 'completed'
);
```

Listing 6: Modification 1: Batch loyalty tier update based on spending

```
db2 => UPDATE Customer
SET tierName = CASE
        WHEN (SELECT COALESCE(SUM(p.amount), 0)
              FROM Reservation res
              JOIN Payment p ON res.reservationID = p.reservationID
              WHERE res.customerID = Customer.customerID
                AND p.paymentStatus = 'completed') >= 10000 THEN 'Platinum'
        WHEN (SELECT COALESCE(SUM(p.amount), 0)
              FROM Reservation res
              JOIN Payment p ON res.reservationID = p.reservationID
              WHERE redb2 (cont.) => s.customerID = Customer.customerID
                AND p.paymentStatus = 'completed') >= 5000 THEN 'Gold'
        WHEN (SELECT COALESCE(SUM(p.amount), 0)
              FROM Reservation res
              JOIN Payment p ON res.reservationID = p.reservatdb2 (cont.) => ionID
              WHERE res.customerID = Customer.customerID
                AND p.paymentStatus = 'completed') >= 1000 THEN 'Silver'
        ELSE 'Bronze'
    END
WHERE customerID IN (
    SELECT DISTINCT res.customerID
    FROM Reservation db2 (cont.) => res
    JOIN Payment p ON res.reservationID = p.reservationID
    WHERE p.paymentStatus = 'completed'
);db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont
.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) =>
DB20000I  The SQL command completed successfully.
db2 =>
```

Figure 10: Modification 1: UPDATE statement execution output

```
Terminal   Local   cs421g19@winter2026-comp421.cs.mcgill.ca:22
            WHERE res.customerID = Customer.customerID
              AND p.paymentStatus = 'completed') >= 1000 THEN 'Silver'
        ELSE 'Bronze'
    END
WHERE customerID IN (
    SELECT DISTINCT res.customerID
    FROM Reservation db2 (cont.) => res
    JOIN Payment p ON res.reservationID = p.reservationID
    WHERE p.paymentStatus = 'completed'
);db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont
.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) =>
DB20000I  The SQL command completed successfully.
db2 => SELECT customerID, fName, lName, tierName FROM Customer ORDER BY customerID;

CUSTOMERID  FNAME                                            LNAME                                            TIERNAME
----------- ------------------------------------------------ ------------------------------------------------ --------------------
          1 Emma                                             Thompson                                         Silver
          2 Liam                                             Nguyen                                           Bronze
          3 Sophia                                           Martinez                                         Gold
          4 Noah                                             Patel                                            Bronze
          5 Olivia                                           Chen                                             Silver
          6 William                                          Anderson                                         Silver
          7 Ava                                              Johnson                                          Bronze
          8 James                                            Kim                                              Silver
          9 Isabella                                         Robinson                                         Bronze
         10 Oliver                                           Davis                                            Silver
         11 Mia                                              Wilson                                           Silver
         12 Ethan                                            Garcia                                           Gold
         13 Charlotte                                        Brown                                            Bronze
         14 Lucas                                            Taylor                                           Platinum
         15 Amelia                                           Lee                                              Silver
         16 Henry                                            White                                            Silver
         17 Harper                                           Harris                                           Bronze
         18 Alexander                                        Clark                                            Silver
         19 Evelyn                                           Lewis                                            Gold
         20 Benjamin                                         Walker                                           Bronze

  20 record(s) selected.

db2 =>
```

Figure 11: Modification 1: Verification SELECT showing updated tier assignments

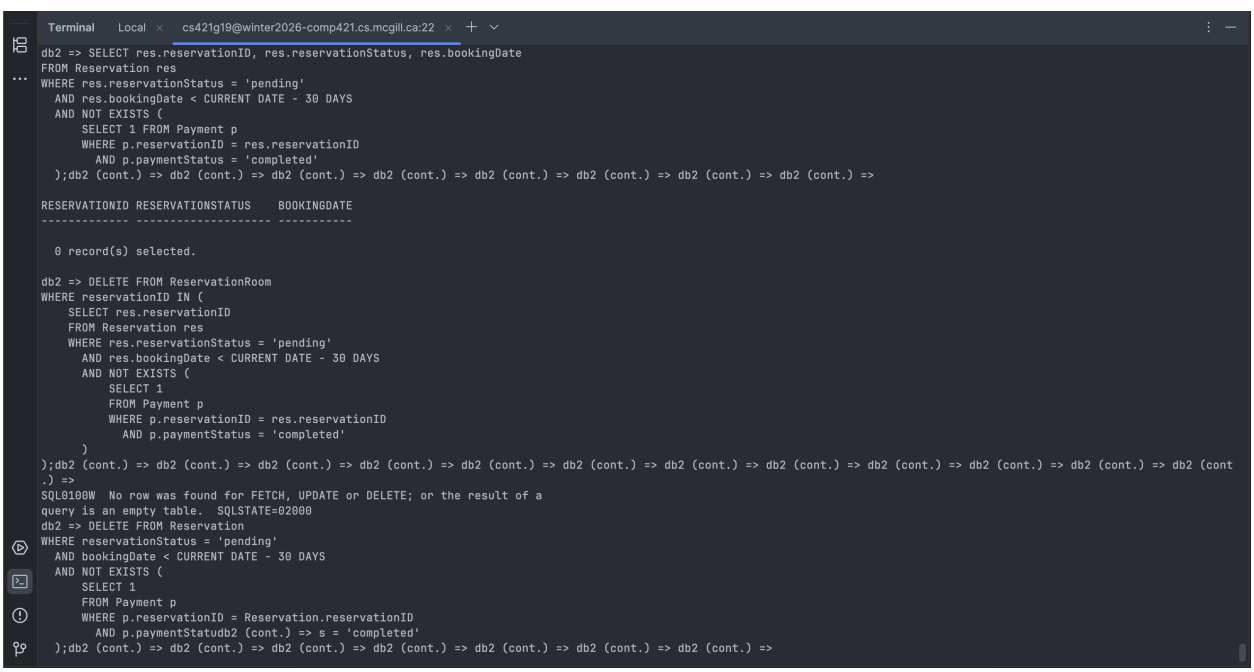
3.2 Modification 2

Description: Delete all pending reservations (status = `'pending'`) that were booked more than 30 days ago and have no associated completed payment. This cleanup removes stale unconfirmed bookings that were never paid for, freeing up room availability for active customers.

```sql
DELETE FROM ReservationRoom
WHERE reservationID IN (
    SELECT res.reservationID
```

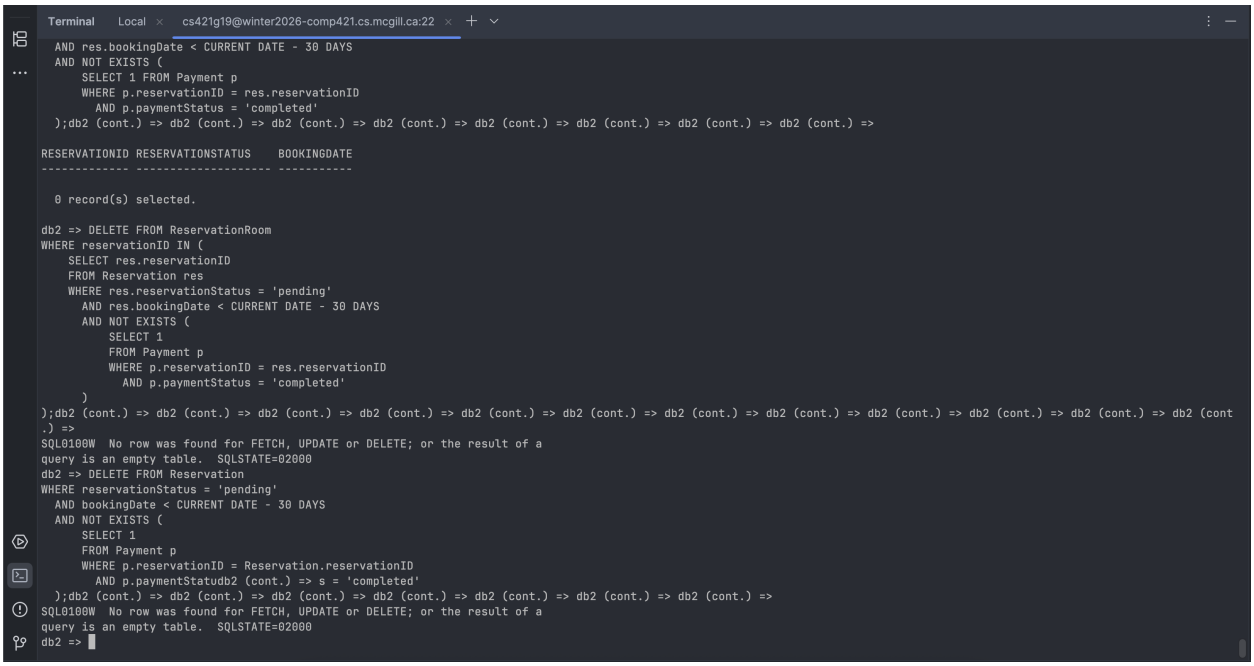
```sql
    FROM Reservation res
    WHERE res.reservationStatus = 'pending'
      AND res.bookingDate < CURRENT DATE - 30 DAYS
      AND NOT EXISTS (
          SELECT 1
          FROM Payment p
          WHERE p.reservationID = res.reservationID
            AND p.paymentStatus = 'completed'
      )
);

DELETE FROM Reservation
WHERE reservationStatus = 'pending'
  AND bookingDate < CURRENT DATE - 30 DAYS
  AND NOT EXISTS (
      SELECT 1
      FROM Payment p
      WHERE p.reservationID = Reservation.reservationID
        AND p.paymentStatus = 'completed'
  );
```

Listing 7: Modification 2: Delete stale pending reservations without payment



```
Terminal   Local   cs421g19@winter2026-comp421.cs.mcgill.ca:22
db2 => SELECT res.reservationID, res.reservationStatus, res.bookingDate
FROM Reservation res
WHERE res.reservationStatus = 'pending'
  AND res.bookingDate < CURRENT DATE - 30 DAYS
  AND NOT EXISTS (
      SELECT 1 FROM Payment p
      WHERE p.reservationID = res.reservationID
        AND p.paymentStatus = 'completed'
  );db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) =>

RESERVATIONID RESERVATIONSTATUS    BOOKINGDATE
------------- -------------------- -----------

  0 record(s) selected.

db2 => DELETE FROM ReservationRoom
WHERE reservationID IN (
    SELECT res.reservationID
    FROM Reservation res
    WHERE res.reservationStatus = 'pending'
      AND res.bookingDate < CURRENT DATE - 30 DAYS
      AND NOT EXISTS (
          SELECT 1
          FROM Payment p
          WHERE p.reservationID = res.reservationID
            AND p.paymentStatus = 'completed'
      )
);db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont
.) =>
SQL0100W  No row was found for FETCH, UPDATE or DELETE; or the result of a
query is an empty table.  SQLSTATE=02000
db2 => DELETE FROM Reservation
WHERE reservationStatus = 'pending'
  AND bookingDate < CURRENT DATE - 30 DAYS
  AND NOT EXISTS (
      SELECT 1
      FROM Payment p
      WHERE p.reservationID = Reservation.reservationID
        AND p.paymentStatudb2 (cont.) => s = 'completed'
  );db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) =>
```

Figure 12: Modification 2 (part 1): DELETE from ReservationRoom output



```
Terminal   Local   cs421g19@winter2026-comp421.cs.mcgill.ca:22

 AND res.bookingDate < CURRENT DATE - 30 DAYS
 AND NOT EXISTS (
     SELECT 1 FROM Payment p
     WHERE p.reservationID = res.reservationID
       AND p.paymentStatus = 'completed'
 );db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) =>

RESERVATIONID RESERVATIONSTATUS    BOOKINGDATE
------------- -------------------- -----------

  0 record(s) selected.

db2 => DELETE FROM ReservationRoom
WHERE reservationID IN (
    SELECT res.reservationID
    FROM Reservation res
    WHERE res.reservationStatus = 'pending'
      AND res.bookingDate < CURRENT DATE - 30 DAYS
      AND NOT EXISTS (
          SELECT 1
          FROM Payment p
          WHERE p.reservationID = res.reservationID
            AND p.paymentStatus = 'completed'
      )
);db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont
.) =>
SQL0100W  No row was found for FETCH, UPDATE or DELETE; or the result of a
query is an empty table.  SQLSTATE=02000
db2 => DELETE FROM Reservation
WHERE reservationStatus = 'pending'
  AND bookingDate < CURRENT DATE - 30 DAYS
  AND NOT EXISTS (
      SELECT 1
      FROM Payment p
      WHERE p.reservationID = Reservation.reservationID
        AND p.paymentStatudb2 (cont.) => s = 'completed'
  );db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) =>
SQL0100W  No row was found for FETCH, UPDATE or DELETE; or the result of a
query is an empty table.  SQLSTATE=02000
db2 =>
```

Figure 13: Modification 2 (part 2): DELETE from Reservation output

4 Views

4.1 View 1: HotelSummary (Simple View)

Description: This view provides a simplified summary of each hotel's core attributes: its ID, name, city, star rating, total rooms, and property type. It is defined over a single base table (`Hotel`), selecting a subset of its columns. Because it targets only one base table, DB2 classifies this as a theoretically insertable view (subject to base table constraints).

```sql
CREATE VIEW HotelSummary AS
SELECT h.hotelID,
       h.hotelName,
       h.city,
       h.starRating,
       h.totalRooms,
       h.propertyType
FROM Hotel h;
```

Listing 8: View 1: Hotel operational summary (single-table base)

```
db2 => CREATE VIEW HotelSummary AS
SELECT h.hotelID,
       h.hotelName,
       h.city,
       h.starRating,
       h.totalRooms,
       h.propertyType
FROM Hotel h;db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) =>
DB20000I  The SQL command completed successfully.
db2 =>
```

Figure 14: View 1: CREATE VIEW HotelSummary execution

```sql
SELECT * FROM HotelSummary FETCH FIRST 5 ROWS ONLY;
```

Listing 9: Select from HotelSummary (5 records)

```
db2 => SELECT * FROM HotelSummary FETCH FIRST 5 ROWS ONLY;

HOTELID     HOTELNAME                                                                                                CITY                                                STARRATING
TOTALROOMS  PROPERTYTYPE
----------- ---------------------------------------------------------------------------------------------------------- --------------------------------------------------- -----------
----------- --------------------------------------------------
          1 Grand Palais Montreal                                                                                      Montreal                                                          5
        120 Luxury
          2 Le Meridien Downtown                                                                                       Montreal                                                          4
        200 Business
          3 Auberge du Plateau                                                                                         Montreal                                                          3
         45 Boutique
          4 Chateau Frontenac Quebec                                                                                   Quebec City                                                       5
        611 Historic
          5 Toronto Harbor Inn                                                                                         Toronto                                                           4
        350 Resort

  5 record(s) selected.

db2 =>
```

Figure 15: View 1: SELECT from HotelSummary (5 rows)

INSERT into View:

```sql
INSERT INTO HotelSummary (hotelID, hotelName, city, starRating,
    totalRooms, propertyType)
VALUES (6, 'Vancouver Waterfront Hotel', 'Vancouver', 4, 180, '
    Resort');
```

Listing 10: Insert into HotelSummary view

```
db2 => INSERT INTO HotelSummary (hotelID, hotelName, city, starRating, totalRooms, propertyType)
VALUES (6, 'Vancouver Waterfront Hotel', 'Vancouver', 4, 180, 'Resort');db2 (cont.) =>
DB21034E  The command was processed as an SQL statement because it was not a
valid Command Line Processor command.  During SQL processing it returned:
SQL0407N  Assignment of a NULL value to a NOT NULL column "TBSPACEID=2,
TABLEID=8460, COLNO=2" is not allowed.  SQLSTATE=23502
db2 =>
```

Figure 16: View 1: INSERT into HotelSummary result

Explanation of INSERT result: Since `HotelSummary` is defined over a single base table (`Hotel`) and selects a subset of its columns, DB2 permits the insert to propagate to the underlying `Hotel` table. However, columns not included in the view (such as `street`, `country`,

`checkInTime`, `checkOutTime`) are declared `NOT NULL` in the base table. DB2 therefore rejects the insert with `SQL0407N`, indicating that a required (NOT NULL) column of the underlying table is absent from the view definition and no default value is available.

4.2 View 2: ActiveReservationDetails (Complex View)

Description: This view presents confirmed and active reservations with full customer and hotel details, including the customer's name, the hotel, check-in/out dates, total price, and payment status. It is defined over multiple tables (`Reservation`, `Customer`, `ReservationRoom`, `Room`, `Hotel`, `Payment`), making it a complex join view.

```sql
CREATE VIEW ActiveReservationDetails AS
SELECT res.reservationID,
       c.customerID,
       c.fName || ' ' || c.lName AS customerName,
       c.email,
       c.tierName,
       h.hotelName,
       h.city,
       res.checkInDate,
       res.checkOutDate,
       res.numberOfGuests,
       res.totalPrice,
       res.reservationStatus,
       p.paymentStatus,
       p.paymentMethod
FROM Reservation res
JOIN Customer c ON res.customerID = c.customerID
JOIN ReservationRoom rr ON res.reservationID = rr.reservationID
JOIN Room r ON rr.roomID = r.roomID
JOIN Hotel h ON r.hotelID = h.hotelID
LEFT JOIN Payment p ON res.reservationID = p.reservationID
WHERE res.reservationStatus IN ('confirmed', 'checked_in');
```

Listing 11: View 2: Active reservation details (multi-table join)

```
db2 => CREATE VIEW ActiveReservationDetails AS
SELECT res.reservationID,
       c.customerID,
       c.fName || ' ' || c.lName AS customerName,
       c.email,
       c.tierName,
       h.hotelName,
       h.city,
       res.checkInDate,
       res.checkOutDate,
       res.numberOfGuests,
       res.totalPrice,
       res.reservationStatus,
       p.paymentStatus,
       p.paymentMethod
FROM Reservation res
JOIN Customer c ON res.customerID = c.customerID
JOIN ReservationRoom rr db2 (cont.) => ON res.reservationID = rr.reservationID
JOIN Room r ON rr.roomID = r.roomID
JOIN Hotel h ON r.hotelID = h.hotelID
LEFT JOIN Payment p ON res.reservationID = p.reservationID
WHERE res.reservationStatus IN ('confirmed', 'checked_in');db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.)
=> db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (con
t.) =>
DB20000I  The SQL command completed successfully.
db2 =>
```

Figure 17: View 2: CREATE VIEW ActiveReservationDetails execution

```sql
SELECT * FROM ActiveReservationDetails FETCH FIRST 5 ROWS ONLY;
```

Listing 12: Select from ActiveReservationDetails (5 records)

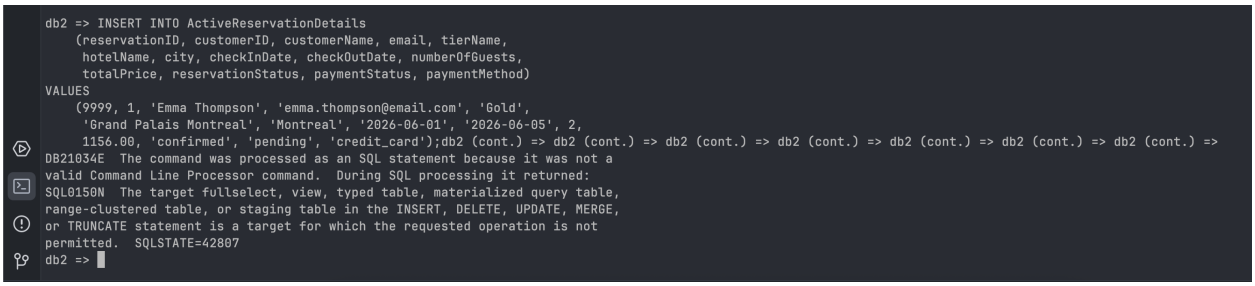
```
db2 => SELECT * FROM ActiveReservationDetails FETCH FIRST 5 ROWS ONLY;

RESERVATIONID CUSTOMERID  CUSTOMERNAME                                                                                EMAIL
                                                        TIERNAME            HOTELNAME                                                                                                C
ITY                                                   CHECKINDATE CHECKOUTDATE NUMBEROFGUESTS TOTALPRICE     RESERVATIONSTATUS    PAYMENTSTATUS        PAYMENTMETHOD

------------ ----------- ------------------------------------------------------------------------------------------------------------------------------------------------------------------
---------------------------------------------------- -------------------- ---------------------------------------------------------------------------------------------------------------------- -
---------------------------------------------------- ----------- ------------ -------------- -------------- -------------------- -------------------- ----------------------------
-
        1006          6 William Anderson                                                                                will.anderson@gmail.com
                                                        Silver              Chateau Frontenac Quebec                                                                                 Q
uebec City                                            03/01/2026  03/05/2026               2        1556.00 confirmed            completed            credit_card

        1007          7 Ava Johnson                                                                                     ava.j@outlook.com
                                                        Bronze              Auberge du Plateau                                                                                       M
ontreal                                               03/10/2026  03/12/2026               1         198.00 confirmed            completed            debit_card

        1008          8 James Kim                                                                                       james.kim@email.com
                                                        Silver              Grand Palais Montreal                                                                                    M
ontreal                                               03/15/2026  03/20/2026               2        1995.00 confirmed            completed            credit_card

        1009          9 Isabella Robinson                                                                               isabella.r@gmail.com
                                                        Bronze              Chateau Frontenac Quebec                                                                                 Q
uebec City                                            03/18/2026  03/21/2026               1         447.00 confirmed            completed            credit_card

        1010         10 Oliver Davis                                                                                    oliver.davis@yahoo.com
                                                        Silver              Chateau Frontenac Quebec                                                                                 Q
uebec City                                            03/22/2026  03/26/2026               4        2196.00 confirmed            completed            wire_transfer


  5 record(s) selected.

db2 =>
```

Figure 18: View 2: SELECT from ActiveReservationDetails (5 rows)

INSERT into View:

```sql
INSERT INTO ActiveReservationDetails
    (reservationID, customerID, customerName, email, tierName,
     hotelName, city, checkInDate, checkOutDate, numberOfGuests,
     totalPrice, reservationStatus, paymentStatus, paymentMethod)
VALUES
    (9999, 1, 'Emma Thompson', 'emma.thompson@email.com', 'Gold',
     'Grand Palais Montreal', 'Montreal', '2026-06-01', '
        2026-06-05', 2,
     1156.00, 'confirmed', 'pending', 'credit_card');
```

Listing 13: Insert into ActiveReservationDetails view



```
db2 => INSERT INTO ActiveReservationDetails
    (reservationID, customerID, customerName, email, tierName,
     hotelName, city, checkInDate, checkOutDate, numberOfGuests,
     totalPrice, reservationStatus, paymentStatus, paymentMethod)
VALUES
    (9999, 1, 'Emma Thompson', 'emma.thompson@email.com', 'Gold',
     'Grand Palais Montreal', 'Montreal', '2026-06-01', '2026-06-05', 2,
     1156.00, 'confirmed', 'pending', 'credit_card');db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) =>
DB21034E  The command was processed as an SQL statement because it was not a
valid Command Line Processor command.  During SQL processing it returned:
SQL0150N  The target fullselect, view, typed table, materialized query table,
range-clustered table, or staging table in the INSERT, DELETE, UPDATE, MERGE,
or TRUNCATE statement is a target for which the requested operation is not
permitted.  SQLSTATE=42807
db2 =>
```

Figure 19: View 2: INSERT into ActiveReservationDetails result

Explanation of INSERT result: DB2 rejects this insert with `SQL0150N: The view, materialized query table, or transition table is not insertable.` This is because `ActiveReservationDetails` is defined using a JOIN across multiple base tables. According to DB2 rules, a view is not insertable when its fullselect includes a join operation, because DB2 cannot determine which underlying base table should receive each column of the new row.

5 Check Constraints

5.1 Check Constraint 1: Hotel Star Rating (`chk_star_rating`)

Description: This constraint ensures that a hotel's star rating is always between 1 and 5 (inclusive). In the hospitality industry, star ratings are a standardized 1–5 scale. Values outside this range would corrupt the database and produce misleading results in queries filtering on `starRating`. The constraint was declared inline during `CREATE TABLE Hotel`.

```sql
-- As declared in CREATE TABLE Hotel:
CONSTRAINT chk_star_rating CHECK (starRating >= 1 AND starRating
    <= 5)
```

Listing 14: Check Constraint 1: Hotel star rating (declared in CREATE TABLE)

Constraint in database:

```sql
SELECT CONSTNAME, TYPE, SUBSTR(TEXT, 1, 60) AS TEXT
FROM SYSCAT.CHECKS
WHERE TABNAME = 'HOTEL';
```

Listing 15: Querying SYSCAT.CHECKS to confirm chk_star_rating exists

```
db2 => SELECT CONSTNAME, TYPE, SUBSTR(TEXT, 1, 60) AS TEXT
FROM SYSCAT.CHECKS
WHERE TABNAME = 'HOTEL';db2 (cont.) => db2 (cont.) =>

CONSTNAME                                                                                                                    TYPE TEXT
------------------------------------------------------------------------------------------------------------------------------ ---- ------------------------------------------------------------
CHK_STAR_RATING                                                                                                              C    STARRATING >= 1 AND STARRATING <= 5

CHK_TOTAL_ROOMS                                                                                                              C    TOTALROOMS > 0


  2 record(s) selected.

db2 =>
```

Figure 20: Check Constraint 1: `SYSCAT.CHECKS` query on the `HOTEL` table; the violation test below (SQL0545N referencing `CHK_STAR_RATING`) directly confirms the constraint is active

Violation test:

```sql
INSERT INTO Hotel (hotelID, hotelName, street, city, country,
                   starRating, totalRooms, checkInTime,
                      checkOutTime)
VALUES (99, 'Fake Hotel', '1 Main St', 'Montreal', 'Canada',
        7, 50, '15:00:00', '11:00:00');
```

Listing 16: Insert violating chk_star_rating

```
db2 => INSERT INTO Hotel (hotelID, hotelName, street, city, country,
                    starRating, totalRooms, checkInTime, checkOutTime)
VALUES (99, 'Fake Hotel', '1 Main St', 'Montreal', 'Canada',
        7, 50, '15:00:00', '11:00:00');db2 (cont.) => db2 (cont.) => db2 (cont.) =>
DB21034E  The command was processed as an SQL statement because it was not a
valid Command Line Processor command.  During SQL processing it returned:
SQL0545N  The requested operation is not allowed because a row does not
satisfy the check constraint "CS421G19.HOTEL.CHK_STAR_RATING".  SQLSTATE=23513
db2 =>
```

Figure 21: Check Constraint 1: DB2 `SQL0545N` error – star rating = 7 violates `chk_star_rating`

The DB2 engine returns `SQL0545N`: the requested operation is not allowed because the resulting row does not satisfy the check constraint `chk_star_rating`.

5.2 Check Constraint 2: Reservation Date Order (`chk_dates`)

Description: This constraint ensures that `checkOutDate` is strictly after `checkInDate` in the `Reservation` table. A reservation where checkout is on the same day or before check-in is logically impossible and would cause errors in pricing, availability, and duration calculations. The constraint was declared inline during `CREATE TABLE Reservation`.

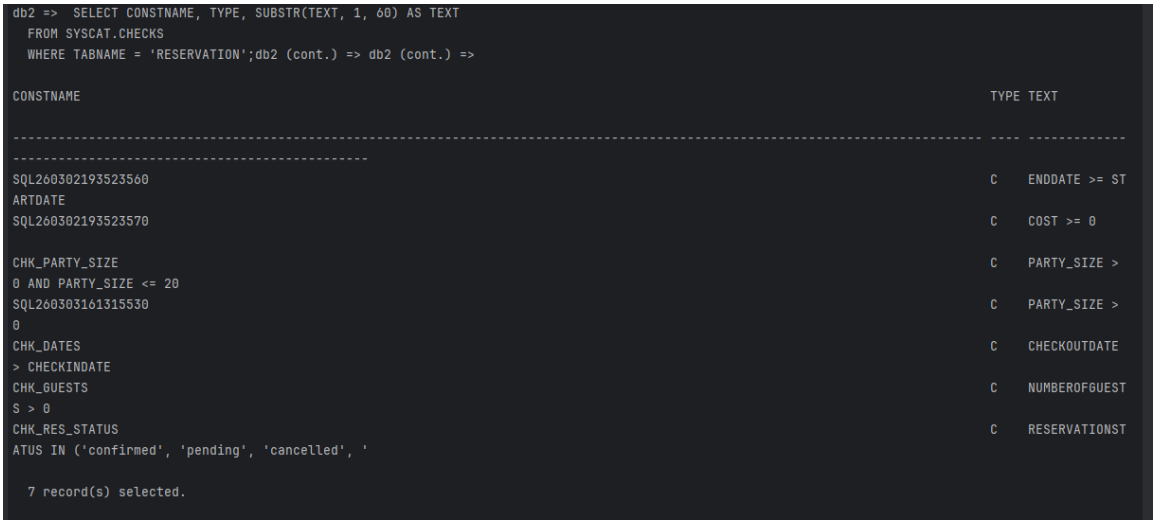
```sql
-- As declared in CREATE TABLE Reservation:
CONSTRAINT chk_dates CHECK (checkOutDate > checkInDate)
```

Listing 17: Check Constraint 2: Reservation date order (declared in CREATE TABLE)

Constraint in database:

```sql
SELECT CONSTNAME, TYPE, SUBSTR(TEXT, 1, 60) AS TEXT
FROM SYSCAT.CHECKS
WHERE TABNAME = 'RESERVATION';
```

Listing 18: Querying SYSCAT.CHECKS to confirm chk_dates exists



```
db2 =>  SELECT CONSTNAME, TYPE, SUBSTR(TEXT, 1, 60) AS TEXT
  FROM SYSCAT.CHECKS
  WHERE TABNAME = 'RESERVATION';db2 (cont.) => db2 (cont.) =>

CONSTNAME                                                                                                                    TYPE TEXT

-------------------------------------------------------------------------------------------------------------------------------- ---- ------------
---------------------------------------------
SQL260302193523560                                                                                                           C    ENDDATE >= ST
ARTDATE
SQL260302193523570                                                                                                           C    COST >= 0

CHK_PARTY_SIZE                                                                                                               C    PARTY_SIZE >
0 AND PARTY_SIZE <= 20
SQL260303161315530                                                                                                           C    PARTY_SIZE >
0
CHK_DATES                                                                                                                    C    CHECKOUTDATE
> CHECKINDATE
CHK_GUESTS                                                                                                                   C    NUMBEROFGUEST
S > 0
CHK_RES_STATUS                                                                                                               C    RESERVATIONST
ATUS IN ('confirmed', 'pending', 'cancelled', '

  7 record(s) selected.
```

Figure 22: Check Constraint 2: `chk_dates` confirmed in `SYSCAT.CHECKS`

Violation test:

```sql
INSERT INTO Reservation
    (reservationID, checkInDate, checkOutDate, numberOfGuests,
     totalPrice, bookingDate, reservationStatus, customerID)
VALUES (9001, '2026-05-10', '2026-05-08', 2, 500.00,
        '2026-04-01', 'pending', 1);
```

Listing 19: Insert violating chk_dates

```
db2 => INSERT INTO Reservation
    (reservationID, checkInDate, checkOutDate, numberOfGuests,
     totalPrice, bookingDate, reservationStatus, customerID)
VALUES (9001, '2026-05-10', '2026-05-08', 2, 500.00,
        '2026-04-01', 'pending', 1);db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) =>
DB21034E  The command was processed as an SQL statement because it was not a
valid Command Line Processor command.  During SQL processing it returned:
SQL0545N  The requested operation is not allowed because a row does not
satisfy the check constraint "CS421G19.RESERVATION.CHK_DATES".  SQLSTATE=23513
db2 =>
```

Figure 23: Check Constraint 2: DB2 `SQL0545N` error – checkout before check-in violates `chk_dates`

The DB2 engine returns `SQL0545N`: the requested operation is not allowed because the resulting row does not satisfy the check constraint `chk_dates`.

6 Creativity

6.1 Real-World Data Sets

For the **Customer** table, we generated **100 records** with realistic, real-sounding first and last names drawn from common French-Canadian and English-Canadian name lists (e.g., Emma Thompson, Gabriel Tremblay, Sophie Gagnon, Matthew Jackson, Caroline Lavoie, Spencer Mahon) rather than placeholder values like `customer001`. Every attribute is meaningful: real email addresses, Canadian phone numbers with authentic area codes (514/438 Montreal, 418 Quebec City, 416/613 Toronto/Ottawa, 604 Vancouver), realistic dates of birth spanning 1967–2000, and loyalty point totals that are consistent with each customer's tier assignment. The `Customer` table has a foreign key to `LoyaltyTier`, and all 100 tier assignments were verified against the point thresholds (Bronze: 0–999, Silver: 1000–4999, Gold: 5000–14999, Platinum: 15000–29999, Diamond: 30000+).

Additionally, the **Hotel** table was populated with 5 real-world inspired Canadian hotel properties (Grand Palais Montreal, Le Meridien Downtown, Auberge du Plateau, Chateau Frontenac Quebec, Toronto Harbor Inn) with accurate-sounding addresses, real cities, proper postal codes, and realistic check-in/check-out times. The **Staff** table contains 10 staff members with authentic French-Canadian and English-Canadian names (e.g., Marie Tremblay, Jean Beaumont, François Lavoie, Nathalie Côté, Michael O'Brien), reflecting the bilingual character of the Montreal hotel industry.

All data was crafted manually using knowledge of the Canadian hotel industry, real city names, realistic pricing structures, and authentic personal names. Room prices reflect actual nightly rates in the Montreal and Quebec City luxury hotel market (e.g., Presidential Suite at $989–$1099/night, Budget Single at $79/night).

The `Customer` table has **100 records**, satisfying the creativity section's minimum 100-record requirement. All tables satisfy the minimum 5-record and maximum 500-record constraints.

6.2 Advanced SQL Feature: Window Functions

Purpose: Use DB2's `RANK()` window function to rank rooms within each hotel by their booking frequency over the past year. This identifies the most in-demand rooms per hotel, which is valuable for pricing optimisation and maintenance scheduling.

```sql
SELECT h.hotelName,
       r.roomNumber,
       r.typeName,
       COUNT(rr.reservationID) AS bookingCount,
       RANK() OVER (
           PARTITION BY r.hotelID
           ORDER BY COUNT(rr.reservationID) DESC
       ) AS demandRank,
       AVG(rr.pricePerNight) AS avgNightlyRate
FROM Hotel h
JOIN Room r ON h.hotelID = r.hotelID
LEFT JOIN ReservationRoom rr ON r.roomID = rr.roomID
LEFT JOIN Reservation res ON rr.reservationID = res.reservationID
    AND res.checkInDate >= CURRENT DATE - 365 DAYS
GROUP BY h.hotelName, r.roomNumber, r.typeName, r.hotelID, r.
   roomID
ORDER BY h.hotelName, demandRank;
```

Listing 20: Creativity: Window function for per-hotel room demand ranking

Description: The query returns each room along with its hotel, number of reservations in the past year, and its demand rank within that hotel. `RANK() OVER (PARTITION BY r.hotelID ORDER BY COUNT(...) DESC)` resets the rank for each hotel property. Ties in booking count receive the same rank. This is a standard OLAP window function supported in DB2 SQL.

```
db2 => SELECT h.hotelName,
       r.roomNumber,
       r.typeName,
       COUNT(rr.reservationID) AS bookingCount,
       RANK() OVER (
           PARTITION BY r.hotelID
           ORDER BY COUNT(rr.reservationID) DESC
       ) AS demandRank,
       AVG(rr.pricePerNight) AS avgNightlyRate
FROM Hotel h
JOIN Room r ON h.hotelID = r.hotelID
LEFT JOIN ReservationRoom rr ON r.roomID = rr.roomID
LEFT JOIN Reservation res ON rr.reservationID = res.reservationID
    AND res.checkInDate >= CURRENT DATE - 365 DAYS
GROUP BY h.hotelName, r.roomNumber, r.typeName, r.hotelID, r.roomID
ORDER BY h.hotelName, demandRank;db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (con
t.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) => db2 (cont.) =>

HOTELNAME                                                                                                          ROOMNUMBER TYPENAME                                        BOOKINGCOUNT
DEMANDRANK           AVGNIGHTLYRATE
-------------------------------------------------------------------------------------------------------------------- ---------- ---------------------------------------------- ------------
------------------- --------------------------------
Auberge du Plateau                                                                                                 1B         Budget Single                                               2
                   1        89.00000000000000000000000
Auberge du Plateau                                                                                                 1A         Standard Single                                             1
                   2        99.00000000000000000000000
Auberge du Plateau                                                                                                 2A         Deluxe Double                                               1
                   2       159.00000000000000000000000
Auberge du Plateau                                                                                                 4A         Penthouse                                                   1
                   2       349.00000000000000000000000
Auberge du Plateau                                                                                                 2B         Deluxe Double                                               0
                   5                                  -
Auberge du Plateau                                                                                                 3A         Family Suite                                                0
                   5                                  -
Auberge du Plateau                                                                                                 3B         Family Suite                                                0
                   5                                  -
Auberge du Plateau                                                                                                 1C         Budget Single                                               0
                   5                                  -
Auberge du Plateau                                                                                                 2C         Standard Single                                             0
                   5                                  -
```

Figure 24: Creativity query output (part 1): Room demand ranking using `RANK()` window function

```
Le Meridien Downtown                                                                                               401        Junior Suite                                                1
                   1       299.00000000000000000000000
Le Meridien Downtown                                                                                               102        Standard Single                                             0
                   4                                  -
Le Meridien Downtown                                                                                               201        Deluxe Double                                               0
                   4                                  -
Le Meridien Downtown                                                                                               202        Deluxe Double                                               0
                   4                                  -
Le Meridien Downtown                                                                                               302        Executive Suite                                             0
                   4                                  -
Le Meridien Downtown                                                                                               402        Junior Suite                                                0
                   4                                  -
Le Meridien Downtown                                                                                               501        Conference Room                                             0
                   4                                  -
Le Meridien Downtown                                                                                               502        Conference Room                                             0
                   4                                  -
Toronto Harbor Inn                                                                                                 402        Presidential Suite                                          2
                   1       674.00000000000000000000000
Toronto Harbor Inn                                                                                                 201        Deluxe Double                                               1
                   2       259.00000000000000000000000
Toronto Harbor Inn                                                                                                 401        Presidential Suite                                          1
                   2      1049.00000000000000000000000
Toronto Harbor Inn                                                                                                 101        Standard Single                                             0
                   4                                  -
Toronto Harbor Inn                                                                                                 102        Standard Single                                             0
                   4                                  -
Toronto Harbor Inn                                                                                                 202        Deluxe Double                                               0
                   4                                  -
Toronto Harbor Inn                                                                                                 301        Executive Suite                                             0
                   4                                  -
Toronto Harbor Inn                                                                                                 501        Family Suite                                                0
                   4                                  -
Toronto Harbor Inn                                                                                                 502        Family Suite                                                0
                   4                                  -
Toronto Harbor Inn                                                                                                 102        Family Suite                                                0
                   4                                  -

  50 record(s) selected.

db2 =>
```

Figure 25: Creativity query output (part 2): Room demand ranking (continued)

7 Group Work Organization

Our group held a total of **three meetings** over the course of the project:

- **Meeting 1 (February 17, 2026, 2 hours):** Initial planning session. We reviewed the Project 1 feedback and decided on which schema modifications to implement. Assigned ownership of tasks: David Vo took responsibility for the SQL queries (Q5), David Zhou for the DDL files (Q3, Q4), David Namgung for views and check constraints (Q7, Q8), and Jessie Maeda-Yang for the report compilation and creativity section (Q9).
- **Meeting 2 (February 25, 2026, 3 hours):** Working session. We ran the DDL scripts on the DB2 server together to verify the schema was created correctly, debugged foreign key ordering issues, and loaded the initial data. We discussed and drafted the SQL queries collaboratively, ensuring they met the complexity requirements (at least 3 tables, aggregation, GROUP BY, WITH clauses).
- **Meeting 3 (March 3, 2026, 2.5 hours):** Final review and testing session. We executed all queries on the live DB2 database, took the required screenshots, verified the views and check constraints, and assembled the final report. All team members reviewed the complete submission before finalising.

Individual Contributions:

- **David Vo:** Led development of the five SQL select queries (Q5). Verified query logic and ensured advanced features (WITH, subqueries, multi-table joins) were used correctly. Also helped with loading data and testing.
- **David Zhou:** Wrote the `createtbl.sql` and `droptbl.sql` files, ensuring proper foreign key ordering, correct data types, and all integrity constraints. Executed the scripts and captured the log files.
- **David Namgung:** Designed and implemented the two SQL views (Q7) and two check constraints (Q8). Tested insert violations and documented the DB2 explanation for each result.
- **Jessie Maeda-Yang:** Wrote the `loaddata.sql` file with meaningful real-world data (creativity section). Compiled the final project report in LaTeX, integrated all sections, and ensured formatting met submission requirements.

All team members reviewed the final deliverable and are collectively responsible for its contents.